

SmartBIS Node Watcher 구축 및 변경 이력

1. 문서 목적

이 문서는 `SmartBIS_Node_Watcher_Build_Log_v1.0.md` 이후 적용된 변경사항을 누적 정리한 문서다. 다음 구축 시 단일 JSON 처리 방식과 배치 단위 처리 방식의 차이를 명확히 이해하고, 전송 완료 판정·다중 콘텐츠·미디어 파일 보관·DB 저장·재처리 절차를 반복해서 설명하지 않도록 하는 것이 목적이다.

2. 최종 처리 구조

외부 시스템은 폐쇄망에서 오픈망 방향으로 파일을 전송한다. 전송 중인 파일을 Node watcher가 즉시 처리하면 JSON 또는 동영상이 아직 완성되지 않은 상태일 수 있으므로, 파일의 생성 이벤트만으로 처리하지 않는다.

최종 구조는 다음과 같다.

```
upload-store/YYYY-MM-DD/BATCH_ID/
├── contents.json
├── 동영상.mp4
├── 이미지.jpg
├── .ready
│   ├── polling으로 폴더와 파일 확인
│   ├── .ready가 있어야 접수 완료
│   ├── 매일 04:00 Asia/Seoul에 일괄 처리
│   ├── JSON의 contents 배열을 content_id별로 검증
│   ├── 미디어 파일을 content-store/YYYY-MM-DD/BATCH_ID로 이동
│   ├── PostgreSQL에 콘텐츠별 1행 UPSERT
│   └── 성공 시 upload-store/archive로 배치 이동
```

3. 단일 JSON 방식에서 배치 방식으로 변경한 이유

기존에는 `upload-store`에 `contents.json` 하나가 생성되면 즉시 처리했다. 이 방식은 다음 문제가 있다.

- 동영상과 이미지가 아직 복사 중이어도 JSON만 먼저 감지할 수 있다.
- 동일한 파일명이 다시 전송되면 이전 파일을 덮어쓸 수 있다.
- JSON 하나에 여러 콘텐츠를 담을 수 없다.
- 처리 완료 파일과 미처리 파일의 구분이 어렵다.
- 폴링 간격이 길면 짧은 파일 이벤트를 놓칠 수 있다.

따라서 파일 단위가 아니라 `BATCH_ID` 폴더 단위로 수신을 확정한다. 전송 업체는 모든 파일 복사가 끝난 후 마지막 단계에서 빈 `.ready` 파일을 생성해야 한다.

4. 권장 수신 규칙

```
upload-store/
└── 2026-09-25/
```

```

└─ BATCH_20260925_0001/
   └─ contents.json
   └─ 장성 관광 홍보 영상.mp4
   └─ 장성 행사 홍보 이미지.jpg
   └─ .ready

```

.ready는 반드시 모든 파일 전송이 끝난 뒤 생성한다. **.ready**가 먼저 생성되면 watcher는 안정성 검사를 수행하더라도 전송 업체의 완료 의도를 신뢰할 수 없으므로, 전송 순서를 협의해야 한다.

5. JSON 형식

JSON 하나에 여러 콘텐츠를 넣을 수 있다.

```

{
  "batch_id": "BATCH_20260925_0001",
  "contents": [
    {
      "content_id": "CNT_BATCH_0001",
      "content_type": "VIDEO",
      "title": "장성 관광 홍보 영상",
      "content": "장성 관광 홍보 영상입니다.",
      "target_file_name": "장성 관광 홍보 영상.mp4",
      "display_start_date": "2026-09-25",
      "display_end_date": "2026-10-01",
      "target_regions": ["장성읍"],
      "template": "행사"
    },
    {
      "content_id": "CNT_BATCH_0002",
      "content_type": "IMAGE",
      "title": "장성 행사 홍보 이미지",
      "content": "장성 행사 안내 이미지입니다.",
      "target_file_name": "장성 행사 홍보 이미지.jpg",
      "display_start_date": null,
      "display_end_date": null,
      "target_regions": null,
      "template": "공지"
    }
  ]
}

```

content_id는 콘텐츠별 고유값이다. **batch_id**는 한 번에 수신된 묶음의 고유값이다. 기간과 지역 제한이 없으면 해당 값은 **null**로 보낸다. JSON 주석(//)은 표준 JSON이 아니므로 실제 전송 파일에는 넣지 않는다.

6. PostgreSQL 저장 방식

JSON의 **contents** 배열 항목마다 DB에 한 행을 저장한다.

content_id	batch_id	content_type
CNT_BATCH_0001	BATCH_20260925_0001	VIDEO
CNT_BATCH_0002	BATCH_20260925_0001	IMAGE

동일한 `content_id`가 다시 수신되면 새 행을 만들지 않고 UPSERT한다. 따라서 콘텐츠 수정 전송은 동일 `content_id`를 유지하고, 신규 콘텐츠만 새로운 `content_id`를 사용해야 한다.

현재 `source_path`는 컨테이너 절대경로가 아니라 `content-store` 기준 상대경로를 저장하는 것을 기준으로 한다.

```
2026-09-25/BATCH_20260925_0001/장성 관광 홍보 영상.mp4
```

이렇게 해야 개발·운영 환경에서 컨테이너 경로가 달라져도 DB 데이터를 수정하지 않고 사용할 수 있다.

7. 미디어 파일 보관

성공한 배치의 미디어는 다음 위치로 이동한다.

```
content-store/YYYY-MM-DD/BATCH_ID/파일명
```

예:

```
content-store/2026-09-25/BATCH_20260925_0001/장성 관광 홍보 영상.mp4
content-store/2026-09-25/BATCH_20260925_0001/장성 행사 홍보 이미지.jpg
```

DB에는 위 상대경로를 기록하고, Spring Boot에는 `content-store`를 읽기 전용으로 연결한다. Node watcher만 업로드 및 이동 권한을 가진다.

8. 전송 완료 판정과 폴링

Windows의 Podman bind mount에서는 파일 변경 이벤트가 안정적으로 전달되지 않을 수 있어 `chokidar`를 polling 모드로 설정했다.

```
usePolling: true,
interval: 500,
awaitWriteFinish: {
  stabilityThreshold: 2000,
  pollInterval: 100
}
```

다만 polling은 전송 완료를 보장하는 기능이 아니다. 따라서 최종 판정은 다음 순서로 한다.

1. 배치 폴더가 존재하는지 확인한다.

2. `contents.json`과 `.ready`가 존재하는지 확인한다.
3. JSON에 선언된 미디어 파일이 모두 존재하는지 확인한다.
4. 파일 크기가 연속 검사에서 변하지 않는지 확인한다.
5. 04:00 일괄 처리 시 검증·DB 저장·미디어 이동을 수행한다.

파일이 04:00 직전에 도착하면 해당 배치는 다음 처리 주기로 넘어갈 수 있다. 이것은 유실이 아니라 안전한 지연 처리다. 관리 업체와 “당일 처리 대상은 04:00 이전에 `.ready` 생성 완료” 규칙을 협의해야 한다.

9. 04:00 일괄 처리

시내버스가 04:00 이후 운영을 시작하고 Android 클라이언트가 운영 시작 시 다운로드하므로, Node watcher는 매일 Asia/Seoul 기준 04:00에 배치를 처리하도록 구성했다.

운영 컨테이너는 계속 실행되며 다음 작업을 수행한다.

- 감시 및 수신 상태 확인
- 04:00 cron 실행
- READY 배치 검색
- 안정성 검사
- JSON 다중 콘텐츠 검증
- PostgreSQL 트랜잭션 저장
- 미디어 이동
- 성공 배치 archive 이동

개발 테스트에서는 다음 환경변수로 즉시 처리한다.

```
podman compose run --rm \
  -e RUN_PROCESS_NOW=true \
  -e STABILITY_CHECK_INTERVAL_MS=1000 \
  -e STABILITY_CHECK_COUNT=1 \
  node-watcher
```

즉시 처리 테스트 컨테이너도 watcher와 동일하게 감시를 시작하므로, 처리 후 종료되지 않는 것이 정상이다. 테스트가 끝나면 `Ctrl+C`로 종료한다.

10. 성공·실패 보관

성공한 배치는 다음으로 이동한다.

```
upload-store/archive/<timestamp>_BATCH_ID/
```

검증 실패 또는 DB 오류 배치는 다음으로 이동한다.

```
upload-store/error/<timestamp>_BATCH_ID/
```

원본을 즉시 삭제하지 않는 이유는 재처리와 장애 원인 분석을 위해서다. archive와 error 보관 기간은 운영 정책으로 별도로 정해야 한다.

11. 해시 및 대용량 파일 메모리 처리

동영상은 `fs.readFile()`로 전체를 메모리에 올리지 않고 `createReadStream()` 기반으로 SHA-256을 계산해야 한다. 현재 Node watcher는 이 방식으로 안정성 검사와 파일 검증을 수행한다.

반면 Android 클라이언트와 Spring Boot 미디어 API도 전체 파일을 메모리에 적재하면 안 된다. Spring Boot는 `Resource` 응답으로 스트리밍하고, Nginx는 다음 설정을 사용한다.

```
proxy_buffering off;
proxy_request_buffering off;
proxy_read_timeout 300s;
```

현재 구현 범위는 파일 다운로드 스트리밍이다. Android에서 동영상 파일을 직접 재생하고 HTTP 재개 재생이 필요해지면 이후 `Range` 요청을 추가한다.

12. 실제 검증 결과

다중 콘텐츠 배치 처리 결과:

```
[database] 배치 저장 성공: BATCH_20260925_0001 (2건)
[archive] 배치 이동 완료
```

DB 확인 결과:

```
CNT_BATCH_0001 | VIDEO | 장성 관광 홍보 영상 | 장성 관광 홍보 영상.mp4
CNT_BATCH_0002 | IMAGE | 장성 행사 홍보 이미지 | 장성 행사 홍보 이미지.jpg
```

기간·지역 제한이 없는 콘텐츠는 다음과 같이 저장된다.

```
display_start_date | null
display_end_date   | null
target_regions     | null
```

13. 다음 구축 시 체크리스트

- ☐ 배치 폴더를 날짜와 `BATCH_ID`로 생성했는가
- ☐ `contents.json`의 `contents` 배열에 모든 콘텐츠를 넣었는가
- ☐ 콘텐츠마다 고유 `content_id`가 있는가
- ☐ 동영상·이미지 파일명이 JSON의 `target_file_name`과 일치하는가
- ☐ 모든 파일 전송 후 마지막에 `.ready`를 생성했는가

- ☐ 실제 JSON에 // 주석을 넣지 않았는가
- ☐ 04:00 처리 규칙을 전송 업체와 협의했는가
- ☐ 동일 콘텐츠 수정 시 `content_id`를 유지했는가
- ☐ 신규 콘텐츠에 새로운 `content_id`를 부여했는가
- ☐ 성공 후 `content-store/YYYY-MM-DD/BATCH_ID`에 파일이 있는가
- ☐ DB의 `source_path`가 상대경로인가
- ☐ 실패 배치가 `upload-store/error`에 보존되는가
- ☐ Spring Boot가 `content-store`를 읽기 전용으로 연결했는가
- ☐ `/api/v1/media/{content_id}` 다운로드 API가 동작하는가

14. 운영상 결론

현재 Node watcher의 핵심 책임은 “파일 전송을 받는 것”이 아니라 “완료가 명시된 배치를 검증하고, 콘텐츠별 DB 행과 미디어 보관 구조를 원자적으로 확정하는 것”이다.

따라서 파일 전송 시스템과 다음 계약을 유지해야 한다.

파일 전체 전송

- `contents.json` 전송
- 모든 파일 검증
- 마지막에 `.ready` 생성
- 다음 04:00 처리

이 규칙을 지키면 polling 간격 때문에 파일을 놓치는 문제를 줄이고, 동일 파일명 덮어쓰기와 전송 중인 대용량 파일의 조기 처리를 방지할 수 있다.

SmartBIS 테스트 문제 및 해결 보장 기록 v1.1

기존 egov-backend, Nginx, Node watcher 문서에 공통으로 추가할 실제 장애 대응 기록이다.

Maven 빌드

호스트에 Maven이 없으면 Maven 컨테이너로 빌드한다.

```
MSYS_NO_PATHCONV=1 podman run --rm \
-v "D:/PROJECT/SmartBIS_jangsung/server/egov-backend:/app" \
-w /app \
docker.io/library/maven:3.9-eclipse-temurin-17 \
mvn clean package -DskipTests
```

Git Bash의 경로 변환을 막지 않으면 `/app`이 `C:/Program Files/Git/app`으로 바뀌어 `workdir` 오류가 발생한다.

Nginx 502 Bad Gateway

실제 원인은 백엔드 컨테이너 재생성 후 IP가 바뀌었지만 Nginx가 이전 IP를 사용한 것이었다.

```
connect() failed (113: Host is unreachable) while connecting to upstream
upstream: http://10.89.0.4:8080/...
```

해결:

```
podman compose restart nginx
```

또는:

```
podman compose up -d --force-recreate egov-backend
podman compose up -d --force-recreate nginx
```

연결 확인:

```
MSYS_NO_PATHCONV=1 podman exec smartbis_jangsung-nginx-1 \
wget -S -O /dev/null \
http://egov-backend:8080/api/actuator/health
```

HTTP/1.1 200이면 Nginx와 백엔드 연결은 정상이다.

Git Bash에 Nginx 설정을 입력한 문제

다음은 Bash 명령이 아니라 `server/nginx/nginx.conf` 파일 내부 설정이다.

```
location /api/ {
    proxy_pass http://egov-backend:8080;
}
```

반영 후:

```
podman exec smartbis_jangsung-nginx-1 nginx -t
```

동영상이 터미널에 출력된 문제

동영상 API에 `-o`가 없으면 바이너리가 터미널에 출력되어 화면이 깨진다. `Ctrl+C`로 중단한다.

Header만 확인:

```
curl -sS -D - -o /dev/null URL
```

파일 저장:

```
curl -sS -o test-video.mp4 URL
```

Podman 포트 전달 문제

컨테이너 내부 호출은 성공하지만 Windows의 `localhost:8443` 연결이 거부될 수 있다.

```
podman machine stop
podman machine set --user-mode-networking=true podman-machine-default
podman machine set --rootful=false podman-machine-default
podman machine start
```

Node watcher의 .ready 문제

`.ready`가 없으면 배치가 있어도 처리하지 않는다. 상위 폴더를 먼저 만든다.

```
mkdir -p upload-store/2026-09-25/BATCH_20260925_0001
touch upload-store/2026-09-25/BATCH_20260925_0001/.ready
```

모든 파일 전송 후 마지막에 `.ready`를 생성해야 한다.

watcher 테스트가 종료되지 않는 문제

watcher는 운영 중 계속 감시하므로 처리 완료 후에도 종료되지 않는 것이 정상이다. 즉시 처리 테스트 후 `Ctrl+C`로 종료한다.

JSON 주석 문제

실제 JSON에는 `// 필수` 같은 주석을 넣으면 안 된다. 주석은 JSON 파싱 오류를 만든다.

미디어 경로 문제

DB `source_path`는 컨테이너 절대경로가 아닌 content-store 기준 상대경로다.

```
2026-09-25/BATCH_20260925_0001/파일명.mp4
```

최종 API 정상 판정

```
HTTP/1.1 200
Content-Type: video/mp4
Content-Length: 원본 파일 크기
Accept-Ranges: bytes
```